Ilton Pfleger Júnior

Um Hipervisor Tipo-1 Leve de Tempo Real para Sistemas Embarcados RISC-V: Projeto e Implementação em C++

Florianópolis
2027

Ilton Pfleger Júnior

Um Hipervisor Tipo-1 Leve de Tempo Real para Sistemas Embarcados RISC-V: Projeto e Implementação em C++

Universidade Federal de Santa Catarina

Orientador: Antônio Augusto Fröhlich

Florianópolis
2027

LISTA DE ILUSTRAÇÕES

Figura 1 – Hipervisor Tipo 1. 9
Figura 2 – Hipervisor Tipo 2. 9
Figura 3 – Níveis de Privilégio RISC-V. 11

SUMÁRIO

1 INTRODUÇÃO 4

2 FUNDAMENTAÇÃO TEÓRICA 5
2.1 EPOS 5
2.2 Interrupções 5
2.3 Exceções 6
2.4 Driver de Dispositivo 6
2.5 Virtualização 6
2.6 *VirtIO* 7
2.7 Diferenças Entre Virtualização e Emulação 7
2.8 Hipervisors 8
2.8.1 Tipo 1 - Bare Metal 8
2.8.2 Tipo 2 9
2.9 Interceptação e Emulação 10
2.10 RISC-V 10
2.10.1 Extensões 10
2.10.2 Níveis de Privilégio 11
2.10.3 *CSRs* 12

3 METODOLOGIA 13
3.1 Virtualização Baseada em Modo Máquina 13
3.2 Modelagem da Troca de Eventos e da Comunicação Entre VMs e VMMs 14
3.2.1 Modelagem Formal da Transformação de Eventos Entre Domínios Distintos 14
3.2.2 Direcionamento e Virtualização de Interrupções Temporais . 15

REFERÊNCIAS 17

1 INTRODUÇÃO

Sistemas críticos estão presentes em uma ampla gama de aplicações, que vão desde dispositivos médicos e sistemas de automação industrial até plataformas automotivas e missões espaciais. Diferentemente da computação de propósito geral, esses sistemas operam sob rigorosos requisitos de tempo real, nos quais a corretude do comportamento não depende apenas da precisão lógica dos resultados produzidos, mas também do momento exato em que esses resultados são disponibilizados. Em muitos casos, atrasos mínimos ou falhas temporais podem comprometer a segurança, a confiabilidade e até mesmo a integridade física de pessoas e equipamentos.

Tradicionalmente, a garantia desse determinismo esteve associada ao uso de plataformas dedicadas, nas quais Sistemas Operacionais de Tempo Real (RTOSs) são empregados para assegurar previsibilidade temporal, isolamento entre tarefas e cumprimento de restrições rígidas de escalonamento. Entretanto, a evolução do software contemporâneo deslocou grande parte dos ecossistemas de desenvolvimento — bibliotecas, frameworks, ferramentas e serviços — para ambientes baseados em Sistemas Operacionais de Propósito Geral (GPOSs), especialmente Linux. Embora esses sistemas ofereçam grande riqueza funcional e amplo suporte da comunidade, sua adoção em contextos críticos demanda esforços adicionais de adaptação, validação e integração.

Como resultado, cresce a necessidade de consolidar, em uma mesma plataforma computacional, cargas de trabalho com requisitos distintos e frequentemente conflitantes: de um lado, aplicações críticas, sensíveis a prazos temporais e dependentes de comportamento determinístico; de outro, aplicações complexas, orientadas a funcionalidades avançadas e dependentes de ambientes generalistas.

Nesse cenário, técnicas de virtualização surgem como uma alternativa promissora, pois permitem a coexistência de múltiplos ambientes de execução sobre o mesmo hardware. Dessa forma, torna-se possível executar aplicações críticas e não críticas de maneira controlada, reduzindo custos de integração e aumentando o aproveitamento da plataforma.

Paralelamente, a arquitetura aberta RISC-V vem ganhando relevância devido à sua flexibilidade, modularidade e independência tecnológica. Seu conjunto de instruções extensível, aliado à ausência de barreiras proprietárias, favorece tanto a pesquisa acadêmica quanto o desenvolvimento industrial, especialmente em soluções customizadas e de baixo custo.

Por isso, este trabalho propõe o projeto e a implementação, em linguagem C++, de uma base de virtualização para sistemas baseados em RISC-V, com foco em previsibilidade temporal, isolamento entre cargas de trabalho e baixo custo de execução. Busca-se, assim, contribuir para a viabilização de plataformas embarcadas capazes de integrar aplicações heterogêneas sem comprometer requisitos críticos de desempenho e confiabilidade.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 EPOS

Embedded Parallel Operating System (EPOS) Fröhlich e Schröeder-Preikschat (1999) é um RTOS orientado a objetos desenvolvido em C++ com foco em aplicações embarcadas e sistemas dedicados. Sua arquitetura é baseada em componentes modulares, permitindo que apenas os recursos necessários para cada aplicação sejam incluídos no sistema final, reduzindo *overheads* desnecessários.

O EPOS adota o conceito de *Application-Oriented Operating System*, no qual um sistema operacional é configurado de acordo com os requisitos específicos de uma aplicação. Para isso, são utilizadas técnicas de orientação a objetos e metaprogramação em toda a base de código desse sistema.

O trabalho desenvolvido neste documento baseia-se fortemente nos princípios arquiteturais do EPOS, especialmente em sua modularidade e abstrações.

2.2 INTERRUPÇÕES

As interrupções constituem-se em um mecanismo fundamental de controle em arquiteturas computacionais, permitindo que o processador suspenda temporariamente a execução de uma tarefa para atender a eventos de alta prioridade. Este paradigma é essencial para a operação assíncrona dos sistemas modernos, viabilizando a reação imediata a sinais de hardware, periféricos ou condições de erro.

Quando uma interrupção é disparada, o fluxo de execução é desviado para uma rotina especializada denominada *Interrupt Service Routine* (ISR). O processo de transição ocorre geralmente em três etapas:

1. **Salvamento de Contexto:** O estado atual da CPU é salvo na pilha ou em uma estrutura de dados específica.

2. **Execução da ISR:** O processador executa o código responsável por tratar o evento que causou a interrupção.

3. **Restauração do Contexto:** O estado anterior é restaurado, permitindo que a tarefa originalmente interrompida retome seu processamento a partir do ponto em que foi suspensa.

2.3 EXCEÇÕES

Exceções são mecanismos de controle semelhantes às interrupções, pois ambos permitem que o processador suspenda temporariamente o fluxo normal de execução para tratar eventos específicos. No entanto, enquanto as interrupções são geralmente causadas por eventos assíncronos externos à CPU — como sinais de periféricos ou temporizadores — as exceções são geradas de forma síncrona pelo processador durante a execução de uma instrução.

Este mecanismo é fundamental para o tratamento de condições anômalas, tais como divisões por zero, acessos inválidos à memória, estouro de pilha ou execução de instruções não suportadas.

Ao detectar uma exceção, o processador desvia o fluxo para uma rotina de tratamento - geralmente a mesma das interrupções. A principal diferença operacional reside no pós-processamento: diferentemente das interrupções, que usualmente retomam o fluxo da tarefa original, o Sistema Operacional (SO) pode decidir encerrar a aplicação que causou a exceção caso ela seja classificada como fatal.

2.4 DRIVER DE DISPOSITIVO

Um driver de dispositivo é um software responsável por permitir a comunicação entre o SO e um componente específico do computador. Ele funciona como uma camada intermediária que traduz os comandos do sistema em instruções compreensíveis para o dispositivo físico, como impressoras, placas de vídeo, teclados, discos rígidos e interfaces de rede.

2.5 VIRTUALIZAÇÃO

Virtualização é uma técnica computacional que permite a execução simultânea e isolada de múltiplos SOs — denominados sistemas convidados — em um mesmo hardware físico. Enquanto um SO convencional atua como uma interface entre o usuário e a máquina, abstraindo a complexidade da arquitetura subjacente, existem sistemas projetados para estender essa abstração a outros SOs. Esses sistemas são denominados hipervisores, e o mecanismo que viabiliza esse processo é chamado de virtualização.

De modo geral, existem dois principais tipos de virtualização: a virtualização completa e a paravirtualização.

Na virtualização completa, também conhecida como *full virtualization*, o SO convidado executa sem ter conhecimento explícito de que está em um ambiente virtualizado. Nesse cenário, operações privilegiadas são interceptadas pelo hipervisor por meio de exceções e tratadas por mecanismos como *trap-and-emulate*. Entretanto, esse processo

pode comprometer o desempenho do sistema, devido ao custo associado às operações de salvamento e restauração de contexto realizadas a cada exceção gerada.

Em contraste, a paravirtualização adota uma abordagem cooperativa entre o SO convidado e o seu hipervisor. Nesse modelo, o sistema convidado é modificado para reconhecer a presença do ambiente virtualizado e, em vez de executar diretamente instruções privilegiadas, realiza chamadas explícitas ao hipervisor, conhecidas como *hypercalls*. Essa estratégia reduz a necessidade de exceções, diminuindo o custo associado ao processo de virtualização.

2.6 *VIRTIO*

VirtIO (RUSSELL, 2008) é um padrão para drivers de dispositivos paravirtualizados, projetado para unificar e otimizar a interface de comunicação entre os sistemas convidados e o hipervisor. Em vez de emular hardwares legados complexos e específicos, o hipervisor expõe uma série de dispositivos virtuais padronizados, permitindo que o SO convidado acesse recursos de rede, armazenamento e console de maneira eficiente e com baixa sobrecarga.

Para viabilizar essa comunicação, o modelo utiliza as chamadas *Virtqueues* — filas circulares compartilhadas responsáveis pelo envio e recebimento das requisições. Dependendo das operações suportadas, cada dispositivo virtual pode alocar uma ou mais dessas filas. Internamente, uma *Virtqueue* divide-se em três componentes principais: uma tabela de descritores (*Descriptor Table*), um anel de índices disponíveis (*Available Ring*) e um anel de índices já utilizados (*Used Ring*).

Na prática, o ciclo de vida de uma operação de Entrada e Saída (IO) ocorre em duas etapas complementares. Inicialmente, quando o sistema convidado deseja realizar uma operação, o dispositivo *VirtIO* escreve os dados na memória compartilhada, insere o descritor correspondente no *Available Ring* e notifica o hipervisor sobre a nova pendência.

Em resposta, o hipervisor consome os descritores desse anel, processa a operação solicitada e registra o resultado no *Used Ring*. Por fim, uma interrupção é enviada ao sistema convidado para sinalizar a conclusão da tarefa. Graças a esse design, múltiplas requisições podem ser tratadas de maneira assíncrona e em lote, o que minimiza as trocas de contexto e reduz expressivamente o custo inerente à virtualização.

Desse modo, o *VirtIO* consolida-se como uma solução fundamental para a infraestrutura de virtualização moderna, pois alcança um equilíbrio entre a flexibilidade de uma interface padronizada e o desempenho da paravirtualização.

2.7 DIFERENÇAS ENTRE VIRTUALIZAÇÃO E EMULAÇÃO

Algumas funcionalidades utilizadas neste trabalho - devido a limitações de hardware - serão emuladas em vez de virtualizadas. A principal diferença entre esses dois

conceitos está na forma em que as operações são executadas.

A emulação é realizada inteiramente por software, reproduzindo o comportamento de uma arquitetura ou dispositivo por meio da interpretação ou tradução de instruções. Entretanto, devido à necessidade de traduzir instruções e simular componentes de hardware, a emulação tende a apresentar menor desempenho em relação a virtualização.

Por outro lado, embora a virtualização também introduza uma camada de software, esta atua primariamente como uma interface de controle e gerenciamento. Isso permite que a maior parte das instruções seja executada diretamente pelo processador físico. Contudo, essa abordagem exige suporte especializado por parte do hardware subjacente, um requisito que nem sempre está disponível em todas as arquiteturas e máquinas.

2.8 HIPERVISORS

Um hipervisor, também conhecido como Monitor de Máquina Virtual (VMM), é uma plataforma que cria e executa máquinas virtuais via virtualização. Segundo Popek e Goldberg (1974) um VMM possuí três características essenciais: primeiro, devem fornecer um ambiente idêntico à máquina original para programas; segundo, os programas executados nesse ambiente, na pior das hipóteses, devem apresentar no máximo pequenas reduções de velocidade que urgem da camada de virtualização; e por último, os VMMs devem deter o contole total sobre os recursos do sistema.

Na visão de (GOLDBERG, 1973), fundamentalmente, existem dois tipos de hipervisores: 1 e 2.

2.8.1 Tipo 1 - Bare Metal

Hipervisores de tipo 1, também conhecidos como bare metal, são aqueles que executam diretamente sobre o hardware físico, sem a necessidade de um sistema operacional hospedeiro intermediário. Por operar diretamente sobre o hardware, esse tipo de hipervisor geralmente apresenta melhor desempenho e menor sobrecarga.

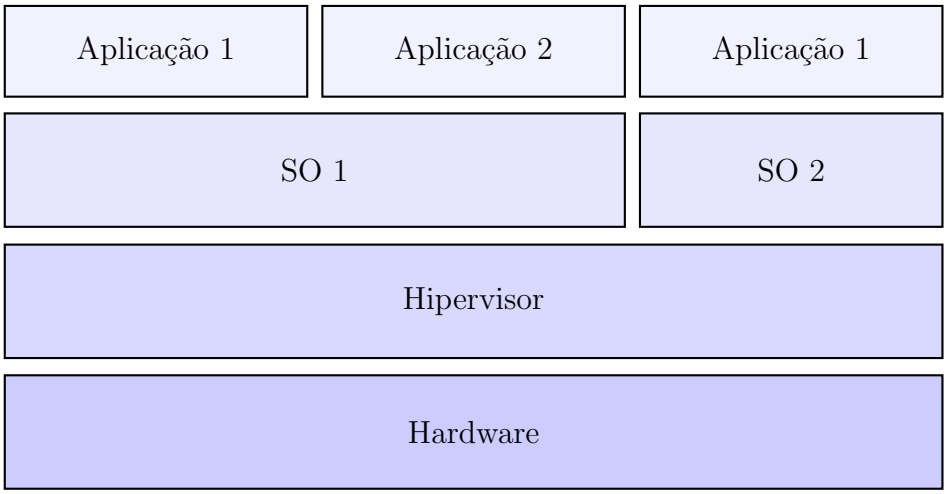


Figura 1 – Hipervisor Tipo 1.

2.8.2 Tipo 2

Diferentemente da solução bare metal, um hipervisor do tipo 2 é executado como um processo dentro de um SO hospedeiro. Porém, apesar de garantir os mesmos recursos do tipo 1, sua arquitetura impõe uma camada intermediária a mais de software, o que restringe a eficiência no acesso aos componentes físicos, gerando uma latência adicional e um menor desempenho.

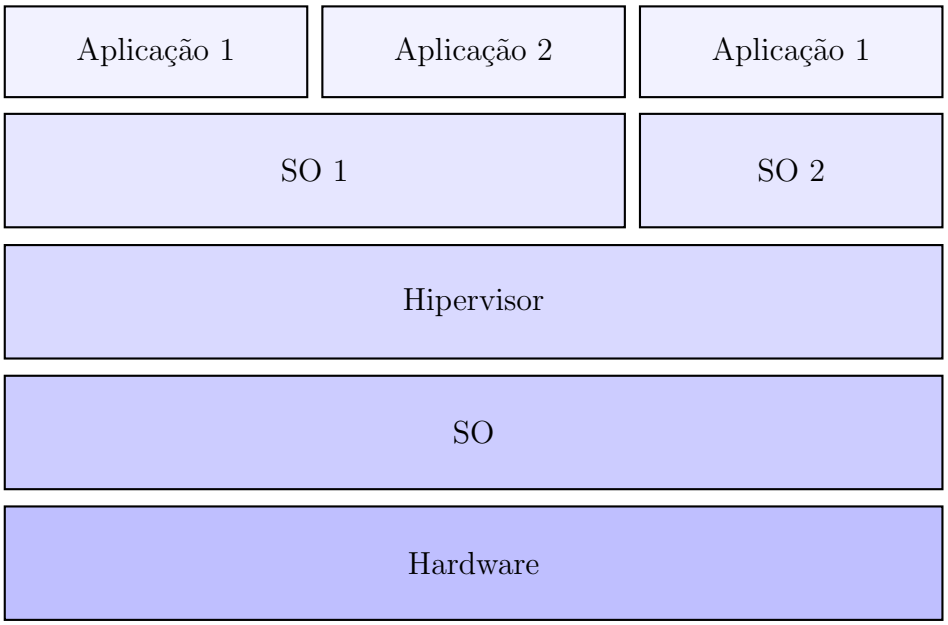


Figura 2 – Hipervisor Tipo 2.

2.9 INTERCEPTAÇÃO E EMULAÇÃO

Do inglês *trap-and-emulate*, interceptação e emulação trata-se de um método utilizado para viabilizar a virtualização em sistemas nos quais o suporte de hardware não é completamente implementado. Nesse modelo, quando a aplicação executada em ambiente virtualizado tenta realizar uma operação que depende de recursos privilegiados, essa operação é interceptada e tratada como uma exceção. O controle é então transferido para um nível de privilégio superior, geralmente o hipervisor, que fica responsável por emular em software o comportamento esperado da instrução ou operação originalmente solicitada.

2.10 RISC-V

A RISC-V é uma Instruction Set Architecture (ISA) baseada nos princípios RISC (Reduced Instruction Set Computer). Diferentemente de arquiteturas proprietárias, a RISC-V possui uma especificação aberta e livre de royalties, o que permite sua utilização tanto em contextos acadêmicos quanto em aplicações comerciais.

Essa ISA define um conjunto básico de instruções, responsável pelas funcionalidades essenciais do processador. No entanto, sua principal característica é a modularidade, permitindo que diferentes extensões sejam incorporadas conforme as necessidades da aplicação. Dessa forma, é possível adaptar a arquitetura para cenários variados, desde sistemas embarcados com restrições de recursos até sistemas de alto desempenho.

2.10.1 Extensões

I Adiciona suporte às instruções básicas de operações com números inteiros. Conforme descrito em (INTERNATIONAL, 2024), essa extensão deve estar presente em qualquer implementação RISC-V, pois define as operações fundamentais de aritmética e manipulação de registradores.

Zicsr Adiciona suporte às instruções de acesso e manipulação de Control and Status Registers (CSRs).

H Adiciona suporte em hardware à virtualização. Porém, apesar de existir, é muito recente e sua implementação em hardware comercial ainda é limitada.

M Adiciona suporte à operações de multiplicação e divisão de números inteiros.

A Adiciona suporte à operações atômicas em memória.

F Adiciona suporte a números em ponto flutuante de precisão simples (32 bits).

D Adiciona suporte a números em ponto flutuante de precisão dupla (64 bits).

C Adiciona suporte à instruções comprimidas de 16 bits.

S Adiciona suporte ao modo supervisor.

G Aglomerado geral de extensões que inclui I, M, A, F, D e C.

Para o escopo deste trabalho, serão usadas apenas as extensões Zicsr, S e G. Os motivos e estratégias adotadas para mitigar e justificar a ausência da extensão H serão discutidas ao longo do texto.

2.10.2 Níveis de Privilégio

Assim como a maioria das arquiteturas modernas, a RISC-V utiliza um modelo hierárquico de privilégios para garantir a segurança e a estabilidade dos sistemas que funcionam sobre ela. Esses níveis de privilégio determinam quais operações e recursos de hardware podem ser acessados por cada contexto de execução.

Por exemplo, ao executar o SO em um nível mais privilegiado, torna-se possível protegê-lo contra interferências de aplicações executadas em níveis inferiores. Dessa forma, o sistema operacional pode impedir tanto que aplicações comprometam seu funcionamento quanto que interfiram entre si, aumentando a segurança do sistema como um todo.

Essa proteção é normalmente implementada por meio de mecanismos de hardware, como registradores privilegiados e regiões de memória protegidas, cujo acesso é restrito de acordo com o nível de privilégio de cada contexto de execução.

O modelo básico da arquitetura estudada neste documento, conforme mostra a Figura 3, contém três níveis de privilégio.

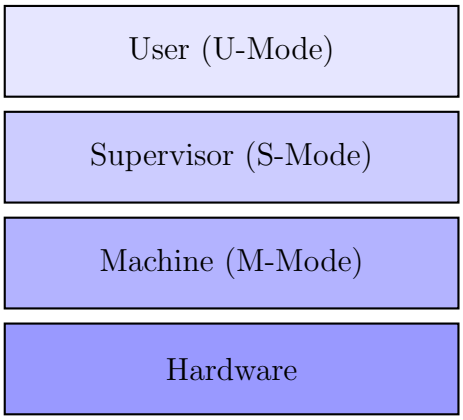


Figura 3 – Níveis de Privilégio RISC-V.

User Mode Nível mais restrito de execução, destinado à operação de aplicações que não devem possuir permissões suficientes para comprometer a integridade, a segurança ou o funcionamento do sistema.

Supervisor Mode Nível utilizado pela maior parte dos SOs modernos, permitindo que eles gerenciem os recursos da máquina e controlem as permissões associadas ao *User Mode*.

Machine Mode Nível mais privilegiado da arquitetura, responsável pelo controle direto do hardware e pela execução de operações críticas de baixo nível. Seus aspectos serão discutidos com maior profundidade ao longo deste texto.

2.10.3 *CSRs*

Os CSRs são registradores especiais utilizados para controlar e monitorar o funcionamento do processador. Esses registradores são responsáveis por armazenar informações relacionadas ao estado da máquina, tratamento de exceções, tratamento de interrupções, configuração de privilégios e controle de funcionalidades específicas da arquitetura.

3 METODOLOGIA

3.1 VIRTUALIZAÇÃO BASEADA EM MODO MÁQUINA

O *Machine Mode* foi concebido originalmente na arquitetura RISC-V como o nível de privilégio mais elevado, responsável pela execução do firmware base e pelo controle direto dos recursos de hardware. O software residente nesse nível encarrega-se da inicialização da plataforma, da configuração dos subsistemas fundamentais e, subsequentemente, da transferência de controle para o SO hospedeiro.

Além das responsabilidades de inicialização, determinadas implementações da arquitetura requerem que o modo máquina forneça serviços privilegiados às camadas superiores. Esses serviços são disponibilizados por meio de interfaces padronizadas, como a Supervisor Binary Interface (SBI), permitindo que sistemas executados em níveis menos privilegiados acessem de forma controlada as funcionalidades de hardware restritas.

No escopo deste trabalho, o M-mode é empregado de forma atípica: como a camada base para a implementação de um hipervisor, visando contornar a ausência da extensão de virtualização (H) na plataforma utilizada. Esta estratégia fundamenta-se na autoridade irrestrita que o modo máquina possui sobre os demais níveis de privilégio da arquitetura, o que viabiliza o controle e a orquestração integral dos SOs convidados. Para tanto, o M-mode dispõe de um conjunto isolado de CSRs e de mecanismos próprios para o tratamento de interrupções e temporização, operando de maneira completamente independente dos modos que operam acima dele.

O isolamento e o controle dos sistemas convidados baseiam-se fortemente no mecanismo de tratamento de exceções da arquitetura. Sempre que o SO virtualizado executa uma instrução privilegiada não permitida ou tenta acessar um recurso restrito, o hardware gera uma exceção que é direcionada nativamente ao modo máquina. Esse comportamento viabiliza a técnica clássica de *trap-and-emulate*, permitindo que o hipervisor intercepte a operação falha e emule o seu comportamento de maneira controlada.

Embora o M-mode careça dos mecanismos de tradução de memória em dois estágios - como a Unidade de Gerenciamento de Memória (MMU) de segundo estágio fornecida pela extensão H -, o isolamento de memória entre os convidados é viabilizado através dos dispositivos de Physical Memory Protection (PMP). Configurado exclusivamente pelo modo máquina, o PMP permite delimitar e proteger regiões específicas da memória física contra acessos indevidos de camadas menos privilegiadas.

Essa granularidade do PMP é também empregada na emulação de *Memory-Mapped I/O* (MMIO), recurso indispensável para o suporte a periféricos virtuais como o *VirtIO*. Esse mecanismo opera mediante a configuração das regiões correspondentes aos dispositivos virtuais como inacessíveis ao sistema convidado; consequentemente, qualquer tentativa

de leitura ou escrita engatilha uma exceção de falha de acesso, permitindo que o hipervisor intercepte a operação e processe a requisição para o SO convidado.

3.2 MODELAGEM DA TROCA DE EVENTOS E DA COMUNICAÇÃO ENTRE Máquinas Virtuais (VMs) E VMMs

3.2.1 Modelagem Formal da Transformação de Eventos Entre Domínios Distintos

A transformação de um evento do domínio do hospedeiro para o domínio dos convidados, e vice-versa, pode ser formalmente modelada como uma composição de funções parciais.

Considere os conjuntos que representam o universo de eventos processáveis em cada domínio:

$$E_{\text{Hypervisor}} = \{\, e \mid e \text{ é um evento gerado ou tratável pelo sistema hospedeiro} \,\} \tag{3.1}$$

$$E_{\text{Virtual}} = \{\, v \mid v \text{ é um evento gerado ou tratável pelo sistema convidado} \,\} \tag{3.2}$$

Define-se, então, a função que maperia um evento $h \in E_{\text{Hypervisor}}$ para um evento $v \in E_{\text{Virtual}}$ como:

$$\mathcal{F} : E_{\text{Hypervisor}} \to E_{\text{Virtual}} \tag{3.3}$$

tal que:

$$\mathcal{F}(h) = \text{Inject}(\text{Translate}(\text{Intercept}(h))), \quad \forall h \in E_{\text{Hypervisor}} \tag{3.4}$$

onde:

- Intercept($\cdot$): representa a captura do evento pelo hipervisor;
- Translate($\cdot$): corresponde à adaptação semântica e estrutural do evento.
- Inject($\cdot$): indica a inserção controlada do evento nos ambientes virtualizados.

Por outro lado, a função que mapeia um evento $v \in E_{\text{Virtual}}$ para um evento $h \in E_{\text{Hypervisor}}$ pode ser definida como:

$$\mathcal{G} : E_{\text{Virtual}} \to E_{\text{Hypervisor}} \tag{3.5}$$

tal que:

$$\mathcal{G}(v) = \text{Dispatch}(\text{Translate}(\text{Intercept}(v))), \quad \forall v \in E_{\text{Virtual}} \tag{3.6}$$

onde:

- Dispatch($\cdot$): representa a submissão operacional de um evento ao hardware hospedeiro.

3.2.2 Direcionamento e Virtualização de Interrupções Temporais

No domínio do hipervisor, eventos temporais resumem-se a interrupções temporais oriundas do hardware de temporização da plataforma. Essas interrupções podem ser transformadas para os convidados através de uma especialização da função de transformação definida anteriormente. Nesse contexto, as interrupções do temporizador — originadas no hardware — são interceptadas pelo hipervisor, processadas pelo subsistema de temporização e posteriormente injetadas no contexto das CPUs virtuais.

O fluxo de transformação dessas interrupções pode ser modelado como uma especialização da função $\mathcal{F}$, restringindo o seu domínio para o subconjunto das interrupções temporais $E_{\text{TemporalHypervisor}} \subset E_{\text{Hypervisor}}$. Para uma interrupção de temporizador $e_t \in E_{\text{TemporalHypervisor}}$, a função de transformação é refinada como $\mathcal{F}_{\mathcal{T}}$, na qual sua definição é dada por:

$$\mathcal{F}_{\mathcal{T}} : E_{\text{TemporalHypervisor}} \rightarrow E_{\text{TemporalVirtual}} \tag{3.7}$$

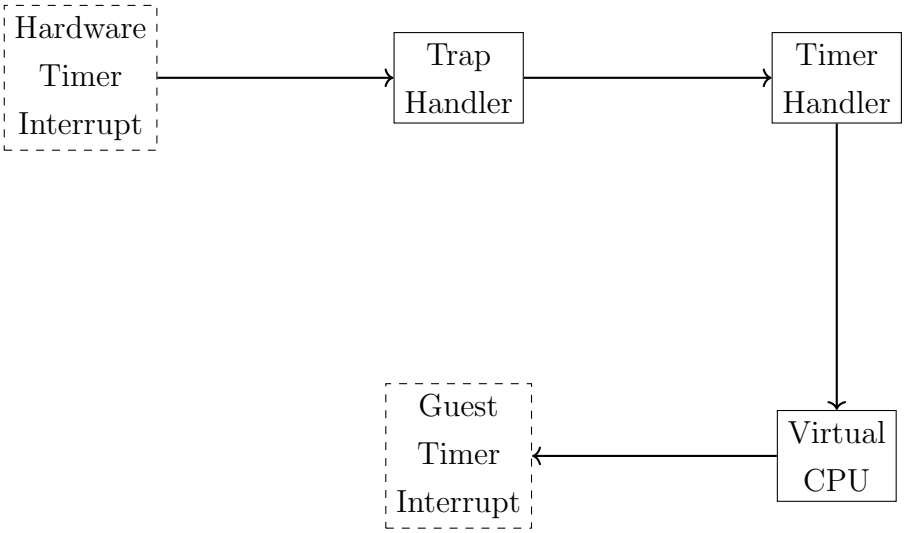
onde:

$$\mathcal{F}_{\mathcal{T}}(e_t) = \text{Inject}_{\text{T}}(\text{Translate}_{\text{T}}(\text{Intercept}_{\text{T}}(e_t))), \quad \forall e_t \in E_{\text{TemporalHypervisor}} \tag{3.8}$$

tal que:

- Intercept$_T(\cdot)$: representa a captura da interrupção pelo hipervisor;
- Translate$_T(\cdot)$: consiste na etapa em que o subsistema responsável pode aplicar alterações no evento, como fatores de escala temporal, viabilizando a execução da VM sob uma abstração de tempo independente do tempo físico da máquina hospedeira;
- Inject$_T(\cdot)$: representa a etapa final na qual o evento é entregue à VM através da geração de uma interrupção virtual.

Na prática, esse processo ocorre em múltiplas etapas. A interrupção é inicialmente gerada pelo hardware físico e interceptada pelo *Trap Handler*. Em seguida, o *Timer* processa a interrupção e a encaminha para a *Virtual CPU*. Por fim, a interrupção é convertida em uma interrupção virtual e injetada no sistema convidado. O fluxo completo desse mecanismo é ilustrado na Figura 3.2.2.


Hardware
Timer
Interrupt
Trap
Handler
Timer
Handler
Guest
Timer
Interrupt
Virtual
CPU

REFERÊNCIAS

FRöHLICH, A.; SCHRöEDER-PREIKSCHAT, W. High performance application-oriented operating systems - the epos approach. In: *Anais do XI Symposium on Computer Architecture and High Performance Computing*. Porto Alegre, RS, Brasil: SBC, 1999. p. 3–9. ISSN 0000-0000. Disponível em: ⟨https://sol.sbc.org.br/index.php/sbac-pad/article/view/19765⟩.

GOLDBERG, R. P. Architectural principles for virtual computer systems. In: . [s.n.], 1973. Disponível em: ⟨https://apps.dtic.mil/sti/tr/pdf/AD0772809.pdf⟩.

INTERNATIONAL, R.-V. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. 2024. ⟨https://docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf⟩.

POPEK, G. J.; GOLDBERG, R. P. Formal requirements for virtualizable third generation architectures. *Commun. ACM*, Association for Computing Machinery, New York, NY, USA, v. 17, n. 7, p. 412–421, jul. 1974. ISSN 0001-0782. Disponível em: ⟨https://doi.org/10.1145/361011.361073⟩.

RUSSELL, R. virtio: towards a de-facto standard for virtual i/o devices. *SIGOPS Oper. Syst. Rev.*, Association for Computing Machinery, New York, NY, USA, v. 42, n. 5, p. 95–103, jul. 2008. ISSN 0163-5980. Disponível em: ⟨https://doi.org/10.1145/1400097.1400108⟩.